

Министерство науки и высшего образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №10

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Поиск расстояний во взвешенном графе»

Выполнили:

студенты групп 24ВВВЗ

Масюк А. Р.

Мамонтов Н. П.

Приняли:

Юрова О. В.

Деев М. В.

Пенза 2025

Цель работы

Освоить на практике методы поиска расстояний во взвешенных графах с использованием алгоритма BFS на языке C++. Закрепить навыки работы с матрицами смежности, очередями из стандартной библиотеки и анализа характеристик графов. **Лабораторное задание**

Задание 1:

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного взвешенного графа G .

Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.

Задание 2:

1. Для каждого из вариантов сгенерированных графов (ориентированного и не ориентированного) определите радиус и диаметр.
2. Определите подмножества периферийных и центральных вершин.

Работа программы

C:\Users\admin\source\repos\IIII10\IIII10\x64\Debug\IIII10.exe

Введите количество вершин графа: 10

Матрица смежности (весов):

0	4	2	9	0	0	0	0	7	0
4	0	2	0	2	10	6	3	5	0
2	2	0	8	6	0	6	0	3	2
9	0	8	0	3	6	0	5	3	0
0	2	6	3	0	10	4	0	9	0
0	10	0	6	10	0	7	6	0	1
0	6	6	0	4	7	0	10	6	3
0	3	0	5	0	6	10	0	3	0
7	5	3	3	9	0	6	3	0	2
0	0	2	0	0	1	3	0	2	0

Матрица смежности взвешенного графа G

```
Введите стартовую вершину для вывода расстояний: 3

Расстояния от вершины 3 до остальных:
До вершины 0: 8
До вершины 1: 5
До вершины 2: 6
До вершины 3: 0
До вершины 4: 3
До вершины 5: 6
До вершины 6: 7
До вершины 7: 5
До вершины 8: 3
До вершины 9: 5

Эксцентриситет вершин:
Вершина 0: 8
Вершина 1: 6
Вершина 2: 6
Вершина 3: 8
Вершина 4: 7
Вершина 5: 7
Вершина 6: 8
Вершина 7: 8
Вершина 8: 6
Вершина 9: 6
```

Процедура поиска расстояний с помощью класса queue

```
Диаметр графа: 8
Радиус графа: 6
Центральные вершины (эксцентриситет = радиусу): 1 2 8 9
Периферийные вершины (эксцентриситет = диаметру): 0 3 6 7
```

Определение радиуса, диаметра, подмножества периферийных и центральных вершин.

Вывод

Освоили на практике методы поиска кратчайших расстояний во взвешенных графах с использованием модифицированного алгоритма обхода в ширину (BFSD). Закрепили навыки работы с очередью `std::queue` из стандартной библиотеки C++ для реализации алгоритма поиска расстояний. Реализовали генерацию взвешенных графов, вычисление эксцентриситетов вершин, определение радиуса, диаметра, центральных и периферийных вершин графа. Написали рабочий код, точно следующий теоретическому описанию алгоритма BFSD и методам анализа характеристик графов.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```

#include <conio.h>
#include <locale.h>
#include <queue>
#include <limits.h>

using namespace std;

#define INF INT_MAX

void BFSWeighted(int** G, int numG, int* dist, int s) {
    queue<int> q;
    int v;

    for (int i = 0; i < numG; i++) {
        dist[i] = INF;
    }

    q.push(s);
    dist[s] = 0;

    printf("\nПуть из вершины %d: ", s);

    while (!q.empty()) {
        v = q.front();
        q.pop();
        printf("%3d", v);

        for (int i = 0; i < numG; i++) {
            if (G[v][i] > 0) {
                int newDist =
                dist[v] + G[v][i];
                if (newDist <
                dist[i]) {
                    dist[i] = newDist;
                    q.push(i);
                }
            }
        }
    }
}

int main() {
    setlocale(LC_ALL, "Russian");

    int** G;
    int** dist;
    int numG;
    int* ecc;
    int vert;

    printf("Введите количество вершин графа: ");
    scanf("%d", &numG);

    ecc = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    dist = (int**)malloc(numG * sizeof(int*));

    for (int i = 0; i < numG; i++) {
        G[i] =
        (int*)malloc(numG * sizeof(int));
        dist[i] =
        (int*)malloc(numG * sizeof(int));
    }

    srand(time(NULL));

    for (int i = 0; i < numG; i++) {
        for (int j = i; j < numG; j++) {
            if (i == j) {
                G[i][j] = 0;
            }
            else {
                G[i][j] = G[j][i] = (rand() % 100 < 70) ? (rand() % 10 + 1) : 0;
            }
        }
    }
}

```

```

    }
}

printf("\nМатрица смежности (весов):\n");
for (int i = 0; i < numG; i++) {    for (int j =
0; j < numG; j++) {
    printf("%4d", G[i][j]);
    }
printf("\n");
}

printf("\nВычисление кратчайших расстояний из каждой вершины:\n");
for (int i = 0; i < numG; i++) {    BFSWeighted(G, numG, dist[i], i);
}

printf("\n\nМатрица расстояний:\n");
for (int i = 0; i < numG; i++) {    for
(int j = 0; j < numG; j++) {        if
(dist[i][j] == INF) {
printf("%4s", "INF");
}
else {
    printf("%4d", dist[i][j]);
}    }
}
printf("\n");
}

printf("\nВведите стартовую вершину для вывода расстояний: ");
scanf("%d", &vert);

if (vert >= 0 && vert < numG) {
printf("\nРасстояния от вершины %d до остальных:\n", vert);
for (int i = 0; i < numG; i++) {
if (dist[vert][i] == INF) {
printf("До вершины %d: недостижима\n", i);
}
else {
printf("До вершины %d: %d\n", i, dist[vert][i]);
}
}
}

printf("\nЭксцентриситет вершин:\n");    for
(int i = 0; i < numG; i++) {        ecc[i] = 0;    for
(int j = 0; j < numG; j++) {            if (dist[i][j] !=
INF && dist[i][j] > ecc[i]) {
ecc[i] = dist[i][j];
}
}
printf("Вершина %d: %d\n", i, ecc[i]);
}

int diameter = 0;
int radius = INF;

for (int i = 0; i < numG; i++) {    if (ecc[i] > diameter &&
ecc[i] != INF) diameter = ecc[i];    if (ecc[i] < radius &&
ecc[i] > 0) radius = ecc[i];
}

printf("\nДиаметр графа: %d\n", diameter);
printf("Радиус графа: %d\n", radius);

printf("Центральные вершины (эксцентриситет = радиусу): ");
for (int i = 0; i < numG; i++) {
if (ecc[i] == radius) {
printf("%d ", i);
}
}
}

```

```
    printf("\nПериферийные вершины (эксцентриситет = диаметру): ");
    for (int i = 0; i < numG; i++) {
        if (ecc[i] == diameter) {
            printf("%d ", i);
        }
    }
    printf("\n");

    for (int i = 0; i < numG; i++) {
        free(G[i]);
    }
    free(dist[i]);
    free(G);
    free(dist);
    free(ecc);

    printf("\nНажмите любую клавишу для выхода...");
    _getch();

    return 0; }
```